ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Java Security: Asymmetric Encryption

Alessandro Marchesano

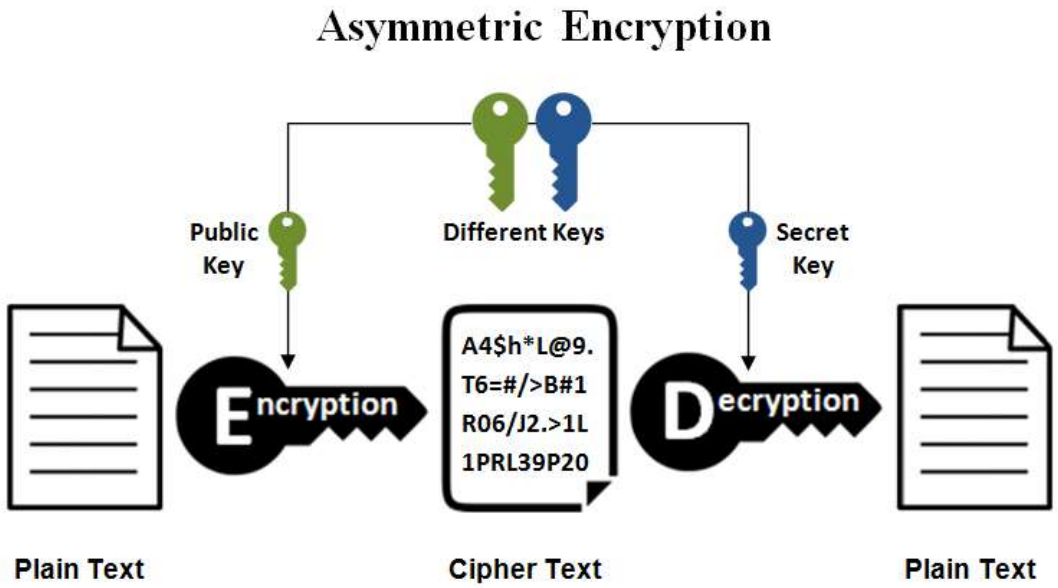
alessandr.marchesano@unibo.it

Asymmetric Encryption in JCE

- In the last lesson, we saw how JCA provides a set of classes, interfaces, and methods for symmetric encryption.
- Of course, the framework exposes the same functionality for **asymmetric encryption** as well.
- Some of the classes we saw for symmetric encryption will still be useful...

Asymmetric Encryption

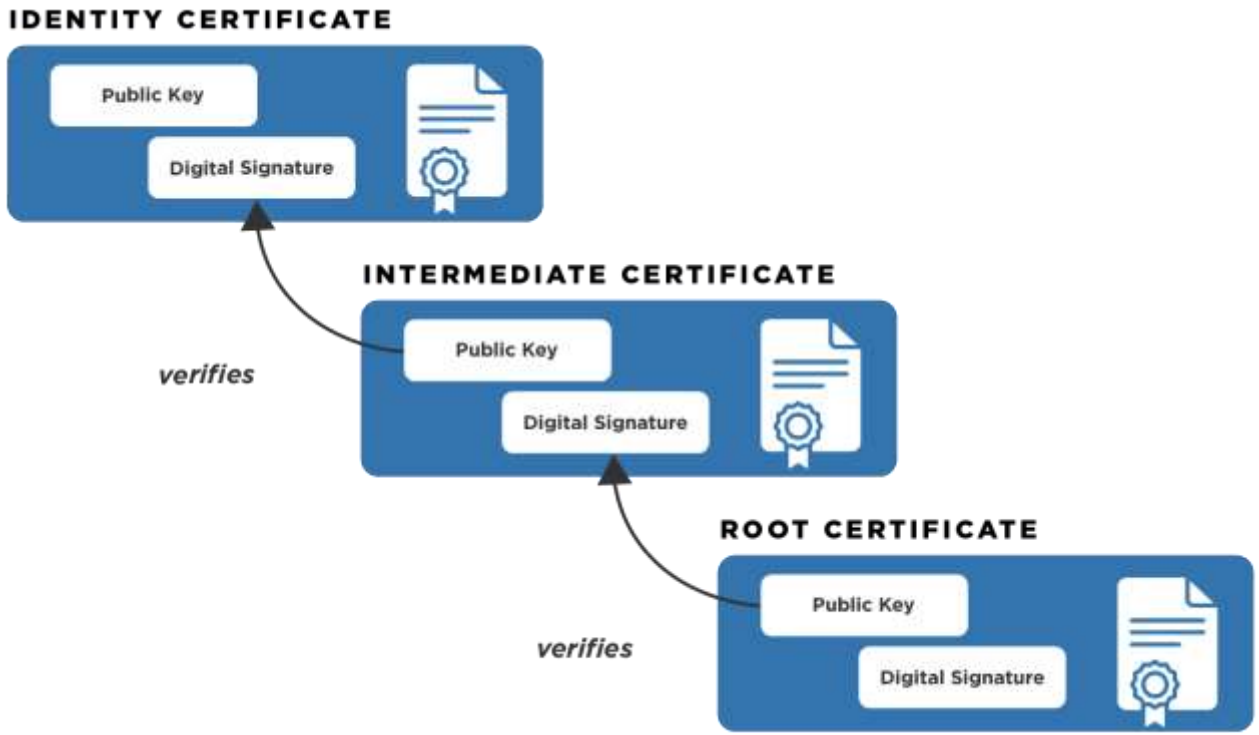
- Asymmetric encryption uses a pair of mathematically related keys - a public key to encrypt data and a private key to decrypt it.
- This method ensures secure communication without sharing secrets, enabling confidentiality and digital signatures.
- It is widely used in HTTPS (SSL/TLS) for secure web browsing and in blockchain technology.



Public Key Infrastructure

- Public Key Infrastructure (PKI) is a framework of technology, policies, and procedures used to manage digital certificates and public-key encryption.

- It binds public keys to entities (users, devices, or servers) through digital certificates issued by trusted Certificate Authorities (CAs), ensuring secure communication, authentication, and data integrity.



Generating Asymmetric Keys

- In order to generate asymmetric keys, we need a `KeyPairGenerator` object.
- It is available via getInstance method:

```
public static KeyPairGenerator getInstance(String
algorithm);
```

- *`KeyPairGenerator kg = KeyPairGenerator.getInstance("RSA");`*

Possible algorithms:

- DiffieHellman
- DSA
- RSA
- RSASSA-PSS
- EC

Initializing KeyPairGenerator

- A key pair generator needs to be initialized before it can generate keys. In most cases, algorithm-independent initialization is sufficient. But in other cases, algorithm-specific initialization can be used.

- `public void initialize(int keysize);`
- `public void initialize(int keysize, SecureRandom sr);`
- `public void initialize(AlgorithmParameterSpec params);`

Obtaining asymmetric keys

- The procedure for generating a key pair is always the same, regardless of initialization (and of the algorithm). You always call the following method from KeyPairGenerator:

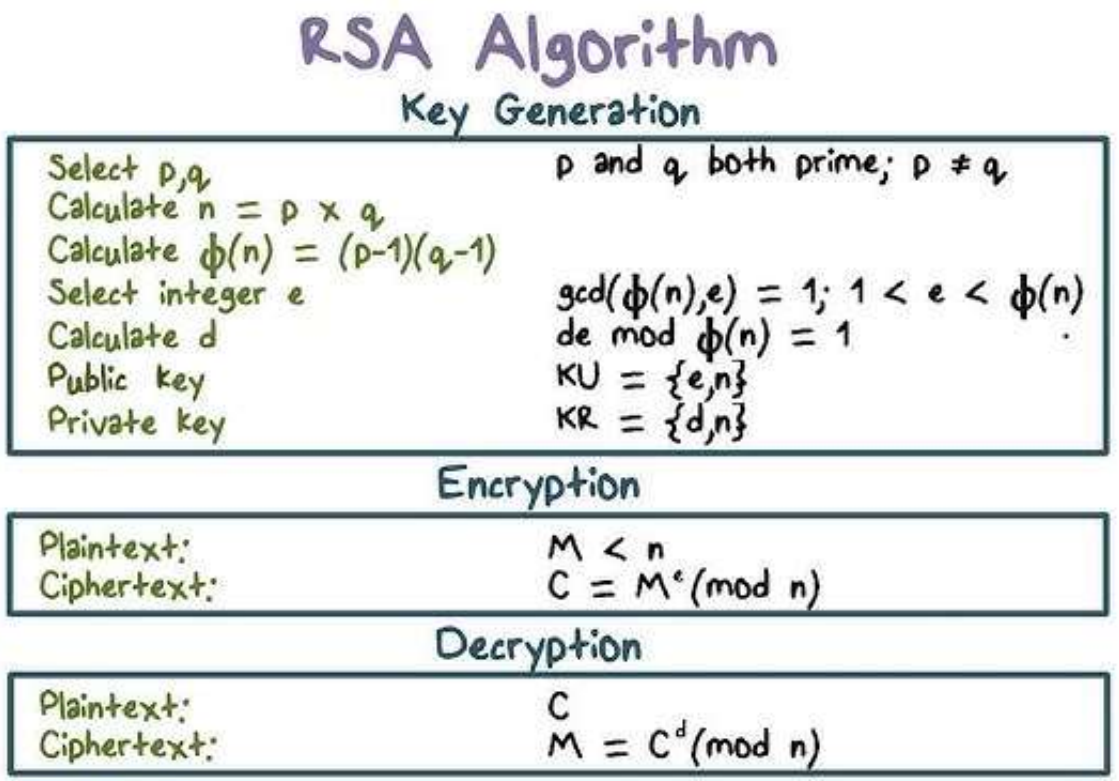
```
KeyPair generateKeyPair()
```

- Multiple calls to generateKeyPair will yield different key pairs.
- You can get the keys simply with:

```
KeyPair pair = keyGen.generateKeyPair();
PrivateKey priv = pair.getPrivate();
PublicKey pub = pair.getPublic();
```

RSA

- Used for encryption and signature on the web.
- The security of RSA is based on the mathematical difficulty of factoring large prime numbers.
- The implementation is optimized to enhance performance (decryption using CRT, e value predefined)



Encrypting with RSA - 1

- We use the Cipher in the exact same way as in the symmetric encryption.

- We must just be careful using "*RSA*" algorithm and specifying the right keys.

```
// Encryption
Cipher cipher = Cipher.getInstance("RSA");
c.init(Cipher.ENCRYPT_MODE, receiverPublicKey);
byte[] encryptedMessage = c.doFinal(message);
// Decryption
c.init(Cipher.DECRYPT_MODE, myPrivateKey);
byte[] decryptedMessage = c.doFinal(message);
```

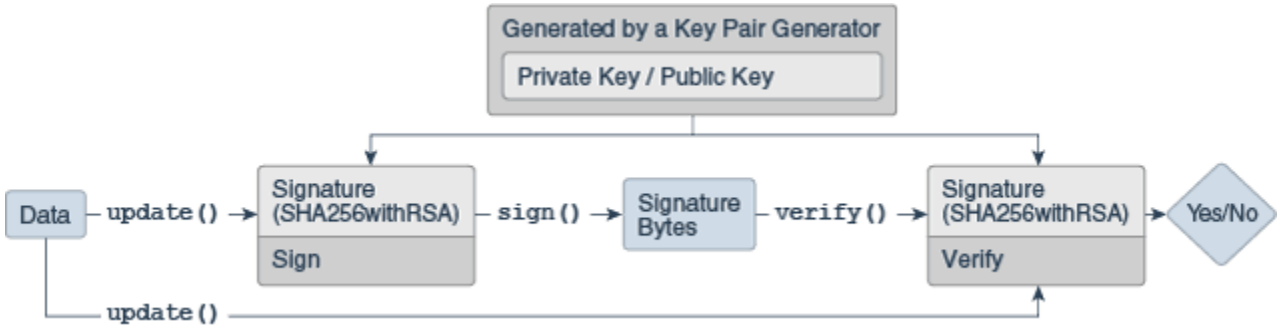
Encrypting with RSA - 2

- Actually, the basic implementation of RSA is susceptible to padding oracle attacks. Therefore, we must use robust versions of these algorithms for greater security (both for encrypting and signing).

```
// Encryption
Cipher cipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
c.init(Cipher.ENCRYPT_MODE, receiverPublicKey);
byte[] encryptedMessage = c.doFinal(message);
// Decryption
c.init(Cipher.DECRYPT_MODE, myPrivateKey);
byte[] decryptedMessage = c.doFinal(message);
```

Signature

- The *`Signature`* class is an engine class designed to provide the functionality of a cryptographic digital signature algorithm that takes arbitrary-sized input and a private key and generates a relatively short string of bytes, called the **signature**, with the following properties:
  - Only the owner of a private/public key pair is able to create a signature. It should be computationally infeasible for anyone having only the public key and a number of signatures to recover the private key.
  - Given the public key corresponding to the private key used to generate the signature, it should be possible to verify the authenticity and integrity of the input.



Getting a Signature object

- We can get a **Signature** object through the *getInstance* method:

```
public static Signature getInstance(String algorithm);
```

Possible algorithms:

NONEwithRSA
MD2withRSA MD5withRSA
SHA1withRSA SHA224withRSA SHA256withRSA SHA384withRSA SHA512withRSA SHA512/224withRSA SHA512/256withRSA
RSASSA-PSS
NONEwithDSA

SHA1withDSA SHA224withDSA SHA256withDSA SHA384withDSA SHA512withDSA
NONEwithECDSA SHA1withECDSA SHA224withECDSA SHA256withECDSA SHA384withECDSA SHA512withECDSA *(ECDSA)*

Choosing the right algorithm for RSA

- Java's provider implementation only offers appendix signing mode.
- You need to install other providers (e.g. BouncyCastle) to have message recovery signing.

- RSASSA-PSS is the most secure implementation of the algorithm.

```
public final void setParameter(AlgorithmParameterSpec
                               params)
```

```
Signature sigSign = Signature.getInstance("RSASSA-PSS");
sigSign.setParameter(new PSSParameterSpec("SHA-256", "MGF1",
                     MGF1ParameterSpec.SHA256, 32,
                     PSSParameterSpec.TRAILER_FIELD_BC));
```

Initializing a Signature object

A *`Signature`* object must be initialized before it is used. The initialization method depends on whether the object is going to be used for signing or for verification.

If it is going to be used for **signing**, the object must first be initialized with the private key of the entity whose signature is going to be generated.

```
final void initSign(PrivateKey privateKey)
```

If instead the Signature object is going to be used for **verification**, it must first be initialized with the public key of the entity whose signature is going to be verified.

```
final void initVerify(PublicKey publicKey)
final void initVerify(Certificate certificate)
```

Using a Signature object

- Like other classes, to use the *Signature* object we must first fill an input array with the *update* method:

```
final void update(byte[] data)
```

- Then, we can sign/verify using the ad-hoc methods:

```
final byte[] sign()
final boolean verify(byte[] signature)
```

RSA Example

```java
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSASSA-PSS");
keyGen.initialize(2048);  // 3072 would be better
KeyPair myPair = keyGen.generateKeyPair();
KeyPair receiverPair = keyGen.generateKeyPair();

PublicKey senderPublicKey = myPair.getPublic();
PrivateKey senderPrivateKey = myPair.getPrivate();
PublicKey receiverPublicKey = receiverPair.getPublic();
PrivateKey receiverPrivateKey = receiverPair.getPrivate();

// SENDER SIDE
// Encrypting a message using the receiver's public key
byte[] message = "Hello!".getBytes();

Cipher c = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
c.init(Cipher.ENCRYPT_MODE, receiverPublicKey);
byte[] encryptedMessage = c.doFinal(message);

// Signing the message with the sender's private key
Signature sigSign = Signature.getInstance("RSASSA-PSS");
sigSign.setParameter(new PSSParameterSpec("SHA-256", "MGF1", MGF1Parame
sigSign.initSign(senderPrivateKey);
sigSign.update(encryptedMessage);
byte[] signature = sigSign.sign();
```

```java
// RECEIVER SIDE
// Verifying the signature using the sender's public key
Signature sigVerify = Signature.getInstance("RSASSA-PSS");
sigVerify.setParameter(new PSSParameterSpec("SHA-256", "MGF1", MGF1Parame
sigVerify.initVerify(senderPublicKey);
sigVerify.update(encryptedMessage);
boolean isVerified = sigVerify.verify(signature);
System.out.println("Signature Verified: " + isVerified);

// Decrypting the message using the receiver's private key
c.init(Cipher.DECRYPT_MODE, receiverPrivateKey);
byte[] decryptedMessage = c.doFinal(encryptedMessage);
System.out.println("Decrypted Message: " + new String(decryptedMessage));
```

```
Signature Verified: true
Decrypted Message: Hello!
```

Limitations of RSA

- A 2048-bit RSA module can encrypt a **maximum of about 245 bytes** with PKCS#1 v1.5 padding **or only 190 bytes** if using the more secure OAEP padding with SHA-256. This makes RSA unusable for documents, images, or large files.

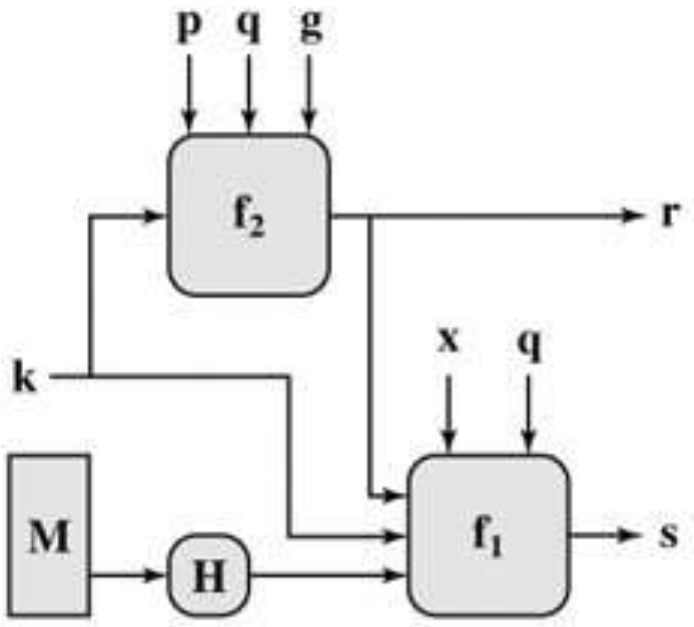
- RSA relies on **extremely CPU-intensive** modular exponentiation operations. Compared to symmetric algorithms like AES, RSA is about 1000 times slower. Encrypting large amounts of data with RSA would quickly saturate system resources.

- RSA is **not** post-quantum secure.

DSA

- The *Digital Signature Algorithm* (DSA) is a standard for digital signatures that uses asymmetric cryptography to ensure message authenticity, integrity, and non-repudiation.

- Relies on the difficulty of calculating **discrete logarithms**.



$s = f_1(H(M), k, x, r, q) = (k^{-1} (H(M) + xr)) \bmod q$

$r = f_2(k, p, q, g) = (g^k \bmod p) \bmod q$

Discrete Logarithm Problem - 1

- I consider a cyclic group over a finite field of 11 elements, where the arithmetic is defined modulo 11.

$$G = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)$$

- This means that 10 + 1 (mod 11) is equal to 11, but 11 + 1 (mod 11) is equal to 1, because the calculation re-starts from the first position.

- In this group, too, I can calculate the powers of the elements:

$$2^1 \mod 11 = 2$$
$$2^2 \mod 11 = 4$$
$$2^3 \mod 11 = 8$$
$$2^4 \mod 11 = 5$$

Discrete Logarithm Problem - 2

- Let's say I now want to find the discrete logarithm of 9 to base 2 modulo 11, which is the corresponding x such that:

$$2^x \equiv 9 \mod 11$$

- I'm proceeding by trial and error because I'm using very small numbers and this is doable:

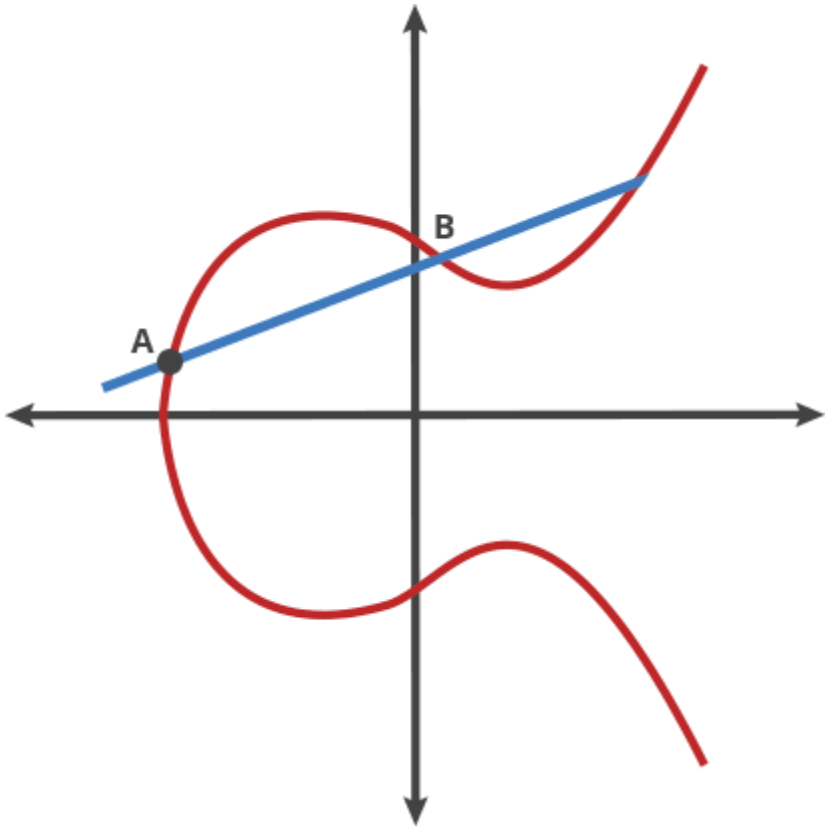
$$2^1 \mod 11 = 2$$
$$2^2 \mod 11 = 4$$
$$2^3 \mod 11 = 8$$
$$2^4 \mod 11 = 5$$
$$2^5 \mod 11 = 10$$
$$2^6 \mod 11 = 9$$

Finding the value of x can be a daunting task, especially with much larger numbers, such as those used in cryptography.

Elliptic Curves

- In simple terms, an elliptic curve is defined by a cubic equation in two variables. The most common form, known as the Weierstrass form, is:

$$y^2 = x^3 + ax + b$$



- **Sum of two points P+Q**: If you draw a line through two points P and Q on the curve, it will intersect the curve at a third point. Reflecting this third point about the x axis, you get the sum P + Q.

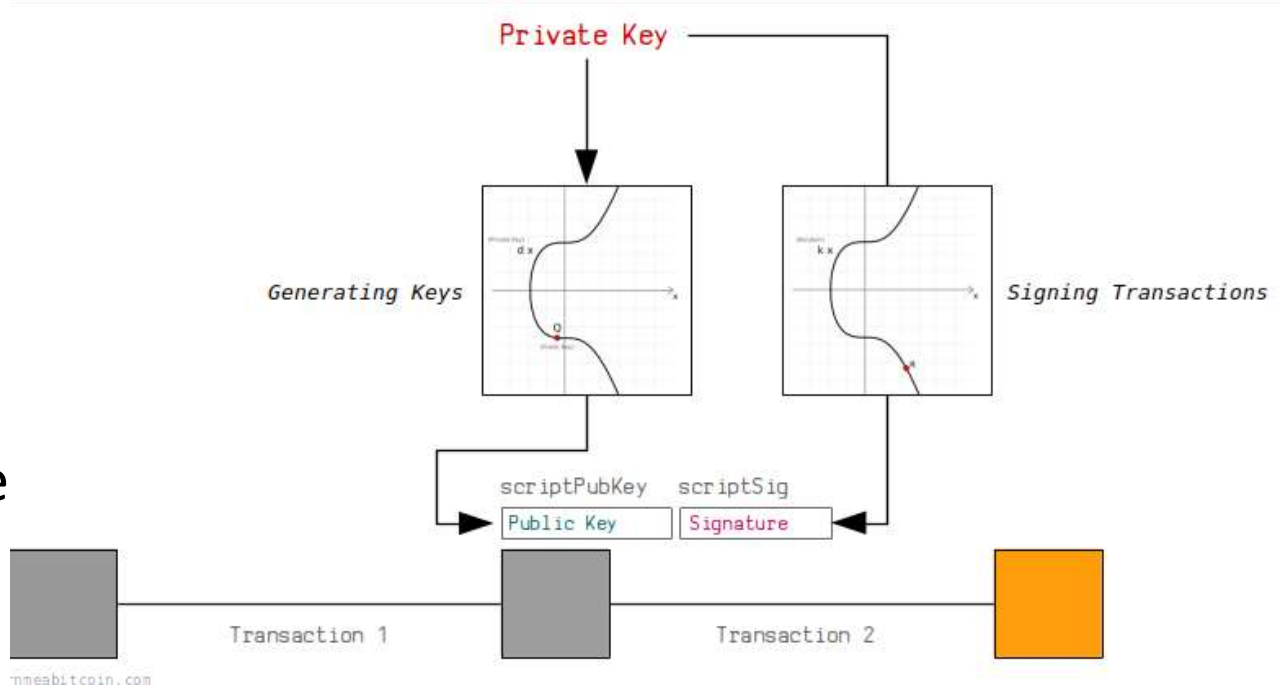
- The discrete logarithm problem, transposed to elliptic curves, becomes a "multiplication" of points on the curve. $Q = kP$

ECDSA

- The **Elliptic Curve Digital Signature Algorithm** (ECDSA) is a public-key cryptographic algorithm used to generate digital signatures.

- It is a variant of DSA that uses the mathematics of elliptic curves to offer the same level of security with much smaller keys and faster calculations.

- A 256-bit ECDSA key offers security comparable to a 3072-bit RSA key.



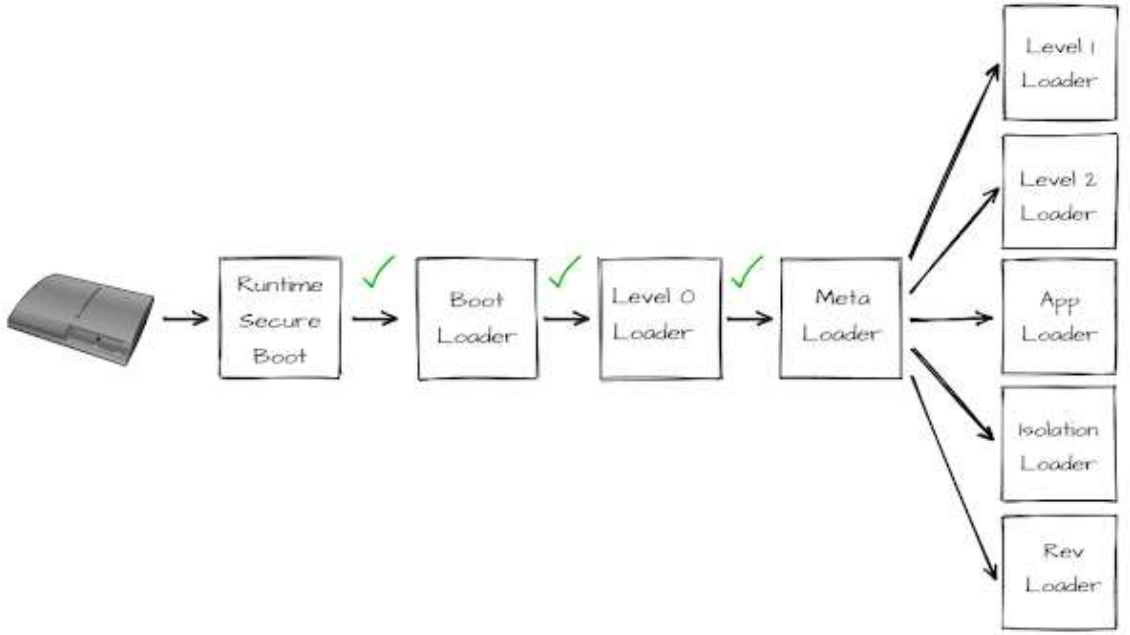
ECDSA Example

```
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("EC");
ECGenParameterSpec ecSpec = new ECGenParameterSpec("secp256r1");
keyGen.initialize(ecSpec, new SecureRandom());
KeyPair senderPair = keyGen.generateKeyPair();

PublicKey senderPublicKey = senderPair.getPublic();
PrivateKey senderPrivateKey = senderPair.getPrivate();

// SENDER SIDE
// Encrypting a message using the receiver's public key
byte[] message = "Hello!".getBytes();

Signature ecdsaSign = Signature.getInstance("SHA256withECDSA");
ecdsaSign.initSign(senderPrivateKey);
ecdsaSign.update(message);
byte[] signature = ecdsaSign.sign();
System.out.println("Firma: " + Base64.getEncoder().encodeToString(signature));
```

```
// RECEIVER SIDE
Signature ecdsaVerify = Signature.getInstance("SHA256withECDSA");
ecdsaVerify.initVerify(senderPublicKey);
ecdsaVerify.update(message);

boolean isCorrect = ecdsaVerify.verify(signature);
System.out.println("Is the signature valid? " + isCorrect);
```

PS3 ECDSA Attack - 1

- The PlayStation 3 operates through a sequence of trust between its components. Each step in the process relies on the assumption that the preceding step's integrity has not been compromised.
- This trust is enforced through ECDSA digital signature with Sony's private key.
- ECDSA signature needs a random nonce for each signature. **Sony used a constant one.**



PS3 ECDSA Attack - 2

- When a message is signed, ECDSA produces two components: (r, s)

- r depends on the random nonce k. s is calculated as:

$$s = k^{-1}(H(m) + d \cdot r) \mod n$$

d is Sony's secret key

- If two messages m1 and m2 are calculated, we obtain:

m1 -> (r, s1)

m2 -> (r, s2)

$$s_1 = k^{-1}(H(m_1) + d \cdot r)$$
$$s_2 = k^{-1}(H(m_2) + d \cdot r)$$

$$k = \frac{H(m_1) - H(m_2)}{s_1 - s_2} \mod n \qquad d = \frac{ks - H(m)}{r} \mod n$$

Asymmetric Encryption in the real world

- Asymmetric Encryption is actually very inefficient. RSA is based on very **computationally intensive mathematics**. In contrast, symmetric encryption relies on bit substitutions and permutations, which are **extremely fast** operations, often accelerated directly by the CPU hardware.

- An asymmetric algorithm cannot encrypt a message larger than its modulus n. In RSA, if you have a 2048-bit key, your message m must be mathematically smaller than n. However, due to padding, the usable space is drastically reduced.

- Better **signing** algorithm exist, but they need to be integrated in **a different scheme**.

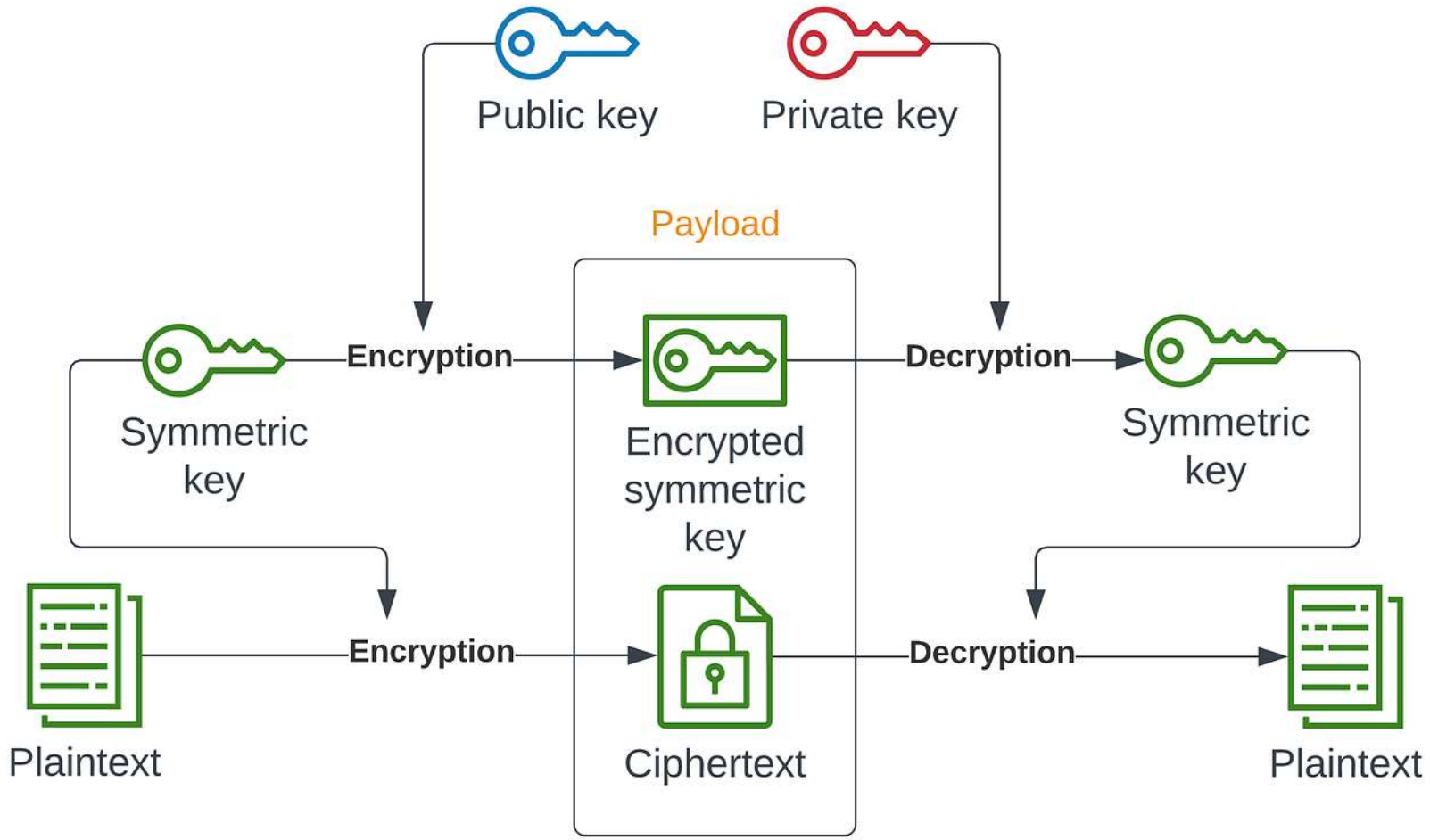
Hybrid Encryption - 1

- **Hybrid encryption** is a method that combines the strengths of symmetric and asymmetric cryptography to protect data efficiently and securely.

- Essentially, it uses the speed of symmetric algorithms (such as AES) to encrypt the actual data and the security of public-key cryptography (such as RSA or ECC) to securely exchange encryption keys.

- Standard in TLS.

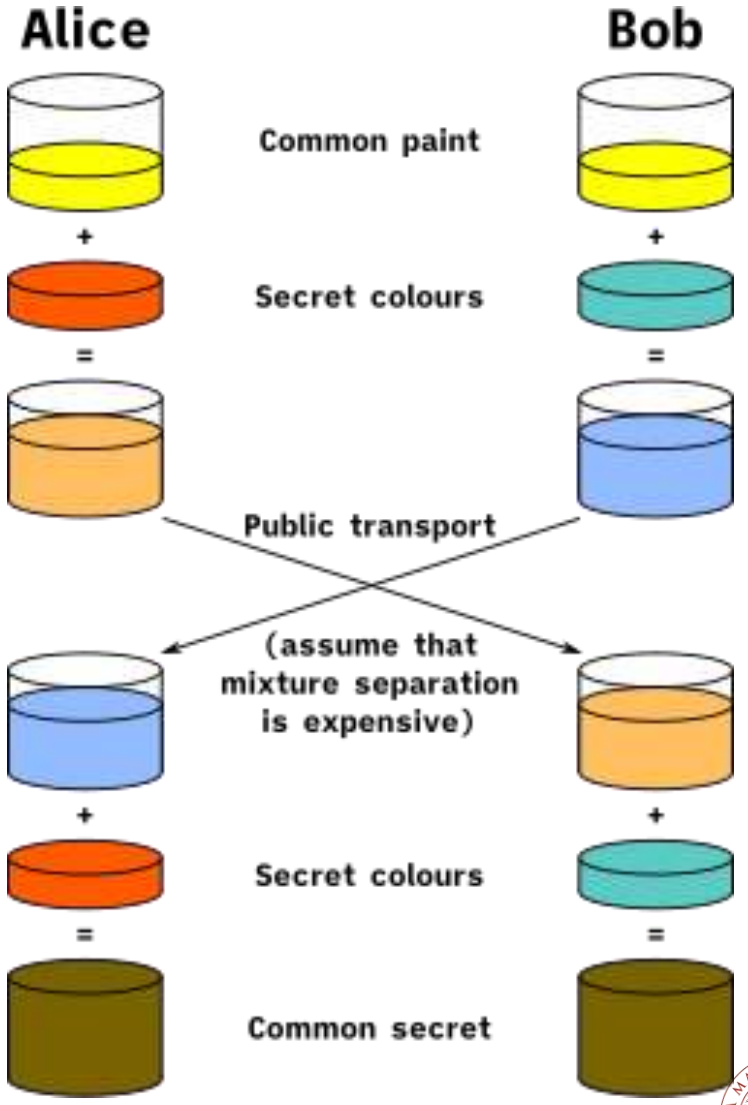
Hybrid Encryption - 2



Diffie-Hellmann

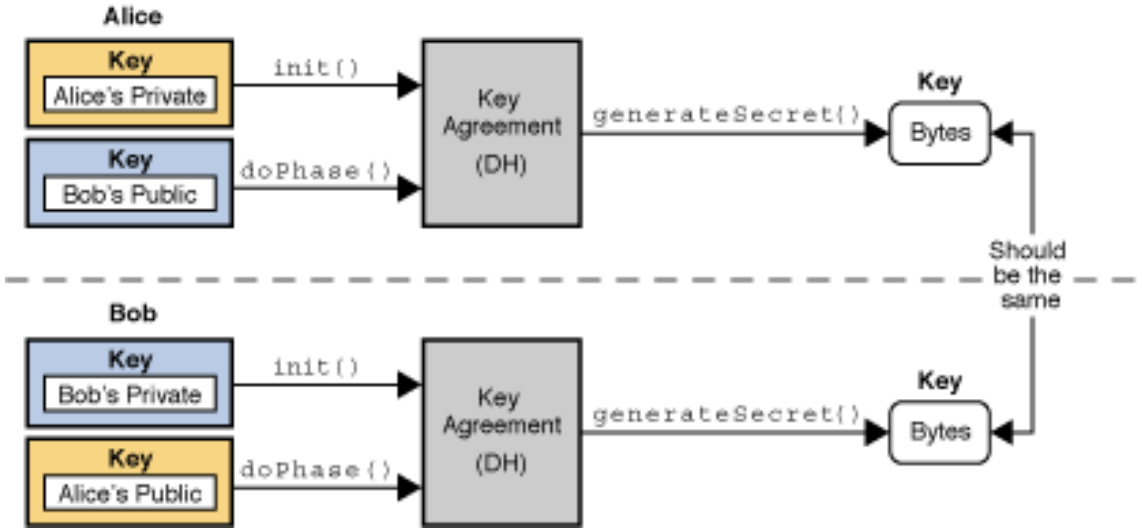
- Diffie-Hellman (DH) key exchange is a cryptographic protocol that allows two parties to generate a **shared secret key** over an insecure communication channel (such as the *Internet*), without the key itself ever being transmitted.

1. Asymmetric: "*Who are you?*"
2. **Diffie-Hellman**: "*How do we talk?*"
3. Symmetric: "*Let's talk!*"



KeyAgreement

- Key agreement is a protocol by which 2 or more parties can establish the same cryptographic keys, without having to exchange any secret information.
- Each party initializes their key agreement object with their private key and then enters the public keys for each party that will participate in the communication.
- When all the public keys have been entered, each KeyAgreement object will generate the same key.



Creating and Initializing a KeyAgreement object

- Obtain a KeyAgremeent object with:

```
public static KeyAgreement getInstance(String algorithm)
```

- Initialize a KeyAgreement object with:

```
public void init(Key key);
```

Possible algorithms:

- DiffieHellman
- ECDH
- ECMQV

Using a KeyAgreement object - 1

- Every key agreement protocol consists of a number of phases that need to be executed by each party involved in the key agreement.

- To execute the next phase in the key agreement, call the doPhase method:

```
public Key doPhase(Key key, boolean lastPhase);
```

- The *key* parameter contains the key to be processed by that phase. In most cases, this is the public key of one of the other parties involved in the key agreement, or an intermediate key that was generated by a previous phase.
- The *lastPhase* parameter specifies whether or not the phase to be executed is the last one in the key agreement.

Using a KeyAgreement object - 2

- After each party has executed all the required key agreement phases, it can compute the shared secret by calling the *generateSecret* method:

```
1. public final byte[] generateSecret()

2. public final SecretKey generateSecret(String algorithm)
```

- The generated secret is a point on a curve or a number in a cyclic group. Although it is secret, its bits are not distributed perfectly randomly (that's why you should not use the second method).
- According to modern standards, a shared secret should never be directly used. It must always go through a KDF (a hashing function is good as well).

Obtain the SecretKey object

- To obtain the secret key we can use **SHA-256** as a KDF.

- Then we can use *`SecretKeySpec`* class to build a *`SecretKey`* object from specifications.

```
public SecretKeySpec(byte[] key, String algorithm)
```

DH Example

```
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("DH");
keyGen.initialize(2048);
KeyPair senderPair = keyGen.generateKeyPair();
PublicKey senderPub = senderPair.getPublic();
PrivateKey senderPriv = senderPair.getPrivate();
KeyPair receiverPair = keyGen.generateKeyPair();
PublicKey receiverPub = receiverPair.getPublic();
PrivateKey receiverPriv = receiverPair.getPrivate()

// Sender side
KeyAgreement ka = KeyAgreement.getInstance("DH");
ka.init(senderPriv);
ka.doPhase(receiverPub, true);
byte[] senderSharedSecret = ka.generateSecret();

MessageDigest hash = MessageDigest.getInstance("SHA-256");
byte[] senderSecretHash = hash.digest(senderSharedSecret);
SecretKeySpec secretKey = new SecretKeySpec(senderSecretHash, "AES");

Cipher c = Cipher.getInstance("AES");
c.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] message = "Hello!".getBytes();
byte[] encryptedMessage = c.doFinal(message);
```

```
// Receiver side
ka.init(receiverPriv);
ka.doPhase(senderPub, true);
byte[] receiverSharedSecret = ka.generateSecret();
byte[] receiverSecretHash = hash.digest(receiverSharedSecret);
SecretKeySpec receiverSecretKey = new SecretKeySpec(receiverSecretHash, "AES");
c.init(Cipher.DECRYPT_MODE, receiverSecretKey);
byte[] decryptedMessage = c.doFinal(encryptedMessage);
System.out.println("Decrypted Message: " + new String(decryptedMessage));
```

```
Decrypted Message: Hello!
```

Why Hybrid Encryption is better

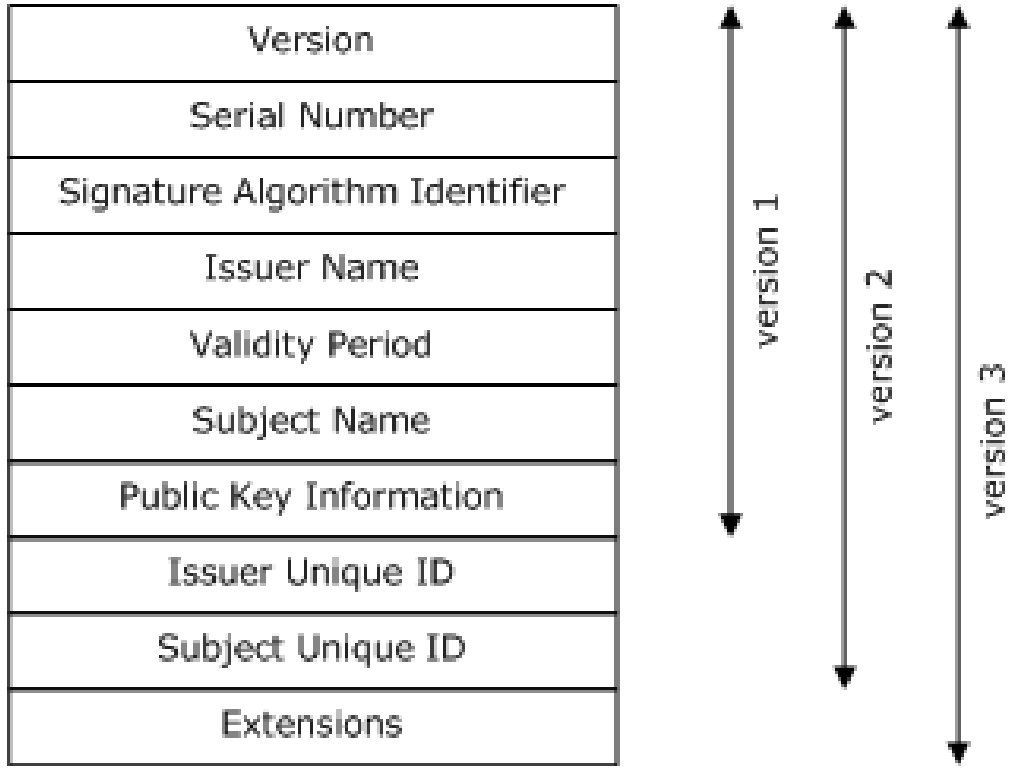
- **High Performance/Efficiency**: Symmetric algorithms like AES are used to encrypt large data payloads, which is much faster than using asymmetric algorithms (e.g., RSA/ECC) for the entire message.

- **Secure Key Distribution**: Asymmetric encryption is used only to encrypt and transfer the symmetric key (session key) securely, solving the challenge of sharing keys over insecure channels.

- **Resilience**: Using ECDH, the symmetric key exists only for that session. If the server's private key is stolen in the future, the attacker cannot decrypt past traffic because the AES session key has never passed over the network and has been erased from RAM.

Certificates

- An X.509 certificate is a digital document that uses the international X.509 Public Key Infrastructure standard to bind a public key to an identity (user, computer, or service).
- It acts as a "digital passport," enabling secure communication, authentication, and encryption for SSL/TLS protocols.
- These certificates are issued and signed by trusted Certificate Authorities (CAs).



CertificateFactory

- The **CertificateFactory** class defines the functionality of a certificate factory, which is used to generate certificate and certificate revocation list (CRL) objects from their encoding.

- CertificateFactory objects are obtained by using one of the *getInstance()* static factory methods.

```
public static final CertificateFactory getInstance(String
                                                    type)
```

The only certificate type available by default is "*X.509*".

Get a Certificate object

- To generate a certificate object and initialize it with the data read from an input stream, use the *`generateCertificate`* method:

```
final Certificate generateCertificate(InputStream inStream)
```

- On a Certificate object, the following methods are available:
  - `PublicKey getPublicKey()` - Gets the public key from this certificate.
  - `void verify(PublicKey key)` - Verifies that this certificate was signed using the private key that corresponds to the specified public key.
  - `void checkValidity()` - Checks that the certificate is currently valid.
  - `X500Principal getSubjectX500Principal()` - Returns the subject value from the certificate.
  - and get all the other values from the certificate.

Certificate Example

```java
FileInputStream fis = new FileInputStream("./Asymmetric/myCert.pem");
CertificateFactory cf = CertificateFactory.getInstance("X.509");
X509Certificate cert = (X509Certificate) cf.generateCertificate(fis);

PublicKey publicKey = cert.getPublicKey();
//System.out.println("Public

cert.verify(publicKey);
System.out.println("Certifica

cert.checkValidity();
System.out.println("Certificate is within its validity period.");

System.out.println("Certificate Subject: " + cert.getSubjectX500Principal());
System.out.println("Certificate Serial Number: " + cert.getSerialNumber());
System.out.println("Certificate Issuer: " + cert.getIssuerX500Principal());
```

```
Certificate is valid.
Certificate is within its validity period.
Certificate Subject: EMAILADDRESS=alessandr.marchesano@unibo.it, CN=Alessandro Marchesano, OU=DISI, O=Alma Ma
er Studiorum Universita di Bologna, L=Bologna, ST=Bologna, C=IT
Certificate Serial Number: 976332502962230532559781978924888654530426958 24
Certificate Issuer: EMAILADDRESS=alessandr.marchesano@unibo.it, CN=Alessandro Marchesano, OU=DISI, O=Alma Mat
r Studiorum Universita di Bologna, L=Bologna, ST=Bologna, C=IT
```

https://github.com/alexmark22/EsercitazioniUNIBO-SicurezzaDellInformazione

Bibliografia

- [1]: JCA Reference Guide - https://docs.oracle.com/en/java/javase/11/security/java-cryptography-architecture-jca-reference-guide.html
- [2]: JCA Standard Algorithm Names - https://docs.oracle.com/javase/8/docs/technotes/guides/security/StandardNames.html
- [3]: Evolution of Signature Algorithms – https://curity.io/blog/the-evolution-of-signature-algorithms/
- [4]: Elliptic Curve Cryptography - https://blog.cloudflare.com/a-relatively-easy-to-understand-primer-on-elliptic-curve-cryptography/
- [5]: ECDSA problems – https://blog.trailofbits.com/2020/06/11/ecdsa-handle-with-care/
- [6]: Psychic Paper – https://arstechnica.com/information-technology/2022/04/major-crypto-blunder-in-java-enables-psychic-paper-forgeries/
- [7]: PS3 Attack - https://deeprnd.medium.com/decoding-the-playstation-3-hack-unraveling-the-ecdsa-random-generator-flaw-e9074a51b831

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Alessandro Marchesano

alessandr.marchesano@unibo.it